

Montenegro Human Development Report — LLM Analysis

Student ID: 23075819 Student Name: Srikar Garlapati

1. Overview and Method

This report describes a pipeline built to read and analyse Montenegro's 142-page National Human Development Report, using three local LLMs **llama3.2:3b**, **qwen2.5vl:7b** and **qwen2.5:3b** running through Ollama where qwen2.5vl:7b is a vision come text model. The aim was not only to summarise the document, but to see how well local models can read a long PDF and pullout summaries, numbers and themes on their own, and how consistent they are with each other while doing so.

Everything is running locally on my own machine not through a paid API or paid compute sources This was a conscious decision, not just a budget one. The source document is a national statistics report, it takes some time to run but nothing to do with my laptop. It also means that the pipeline can be re-run as many times as needed without worrying about the cost for API's and for computing powers.

The pipeline executes in a chain of simple steps, shown in Figure 1 below. First the PDF is loaded and broken into chapters then into smaller recursive overlapping chunks so that local models especially Ollama models can handle the content easily. Next a model reads each chunk and generates chapter summaries, key results, counts of themes, and structured numbers. A second, vision-based model based on qwen looks at the raw page images independently this can take key insights of metrics that might be inside a chart or infographic box rather than a normal sentence. Then a third model looks at the summaries of the first model and scores the accuracy

Finally all three models run the same task and compared side by side .

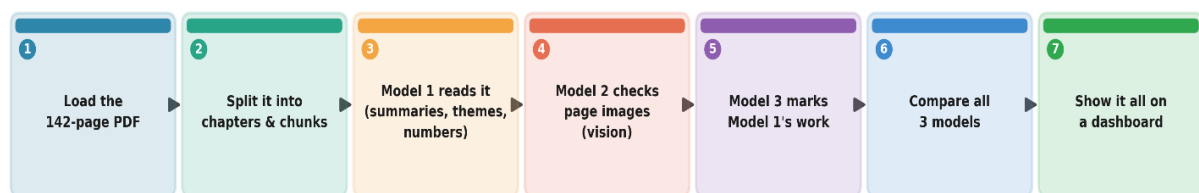


Figure 1. The steps Pipeline follows from start to end.

PDF gets parsed only once and if needed stores all the parsed content to pre-process the data, but from that point, everything downstream (summaries, metrics, themes and all other crucial information) is extracted and generated independently by more than one model, which is what makes the comparison in Section 2.

2. Summarisation Quality and Prompt Quality

Every prompt used in this pipeline instructed to only use the text it was given that means the temperature is set to "0" (No creativity) and don't add anything from outside knowledge. This mattered a lot. none of the three models invented facts about Montenegro that weren't actually in the report even though it's a real country a model might already "know" things about from its own training. Without that instruction, it would have been very easy for a model to quietly fill a gap with a plausible sounding, but wrong number.

This is clearly shown in Figure 2, the three models behaved a little differently from each other. The main text reading model wrote the most overall and was the one working through the full document, so it picked up the widest spread of themes. The second model that read the page images directly was much more conservative — it wrote far less, and only reported a number when it could actually see it printed on the page, rather than guessing at one. The third model, which was trained to fact check the first model's summaries, mostly agreed with what it read, but awarded lower scores to a handful of chapters where additional detail had crept in that wasn't really stated in the original text.

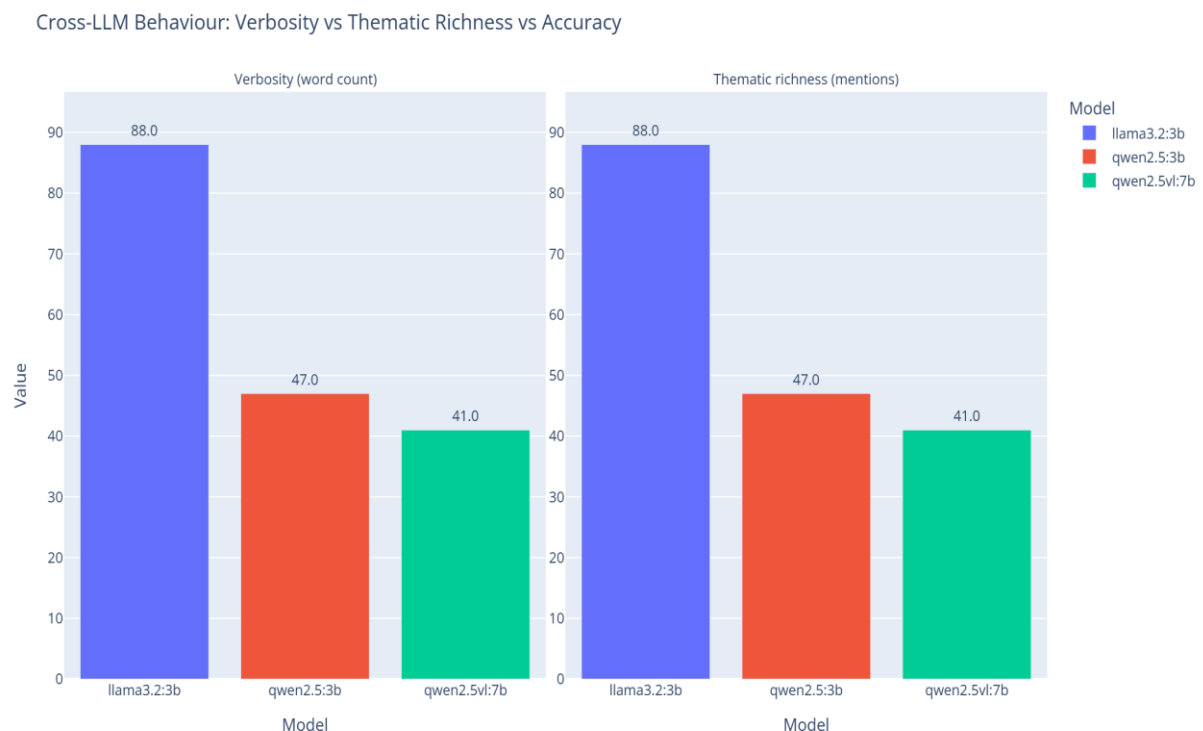


Figure 2. How much each model wrote, and how many themes it picked up, compared side by side.

My own takeaway here is that being strict with a prompt (telling a model exactly what not to do) is often more useful than being clever with it or telling exactly what to do. The models that stuck closest to the instructions gave the most trustworthy output, even when that meant they wrote less. Structured clean and clear prompts and prompting style also played a major role while extracting the data from the reports actually this area has more weightage while playing with LLM's especially local LLM's like Ollama.

3. Quality of Extracted Structured Outputs

All of the extracted values from the report have been stored in JSON format to be reused, reviewed, or used in some other application down the line without having to go through the entire pipeline again. Some of the values that have been located were: the value of HDI index – 0.802, the number of years of life expectancy – 76.2, the poverty risk rate – 30.3% and the level of informal employment – 5%. It is worth noting that all of the information has not been extracted, and in particular, the real GDP growth rate has not been mentioned anywhere.

The two numbers that stood out being suspiciously low were Population and GNI Per Capita. My hypothesis is that there may have been an incomplete extraction of numbers from the text because the algorithm was satisfied with whatever it got in place of the actual answer. In other way, it might have grabbed only some part of a number, say "14" instead of "14,558", and was consent with that answer. As we can see it is possible to miss such mistakes when viewing only the finished dashboard, which serves as another proof that we should verify all the data in the original PDF file because local LLM's are not much stronger when compared to cloud based models which needs more compute power to run .

Topics such as those depicted in Figure 3 were very straightforward. The economy and employment were represented in almost all of the topics, followed by informality. It was no surprise since the entire document is centered on informal work. Topics such as health and climate change did not appear at all. Again, this was understandable since the entire document talks about informal work. It was fascinating to see that statistics make sense since the model is analyzing the data properly.

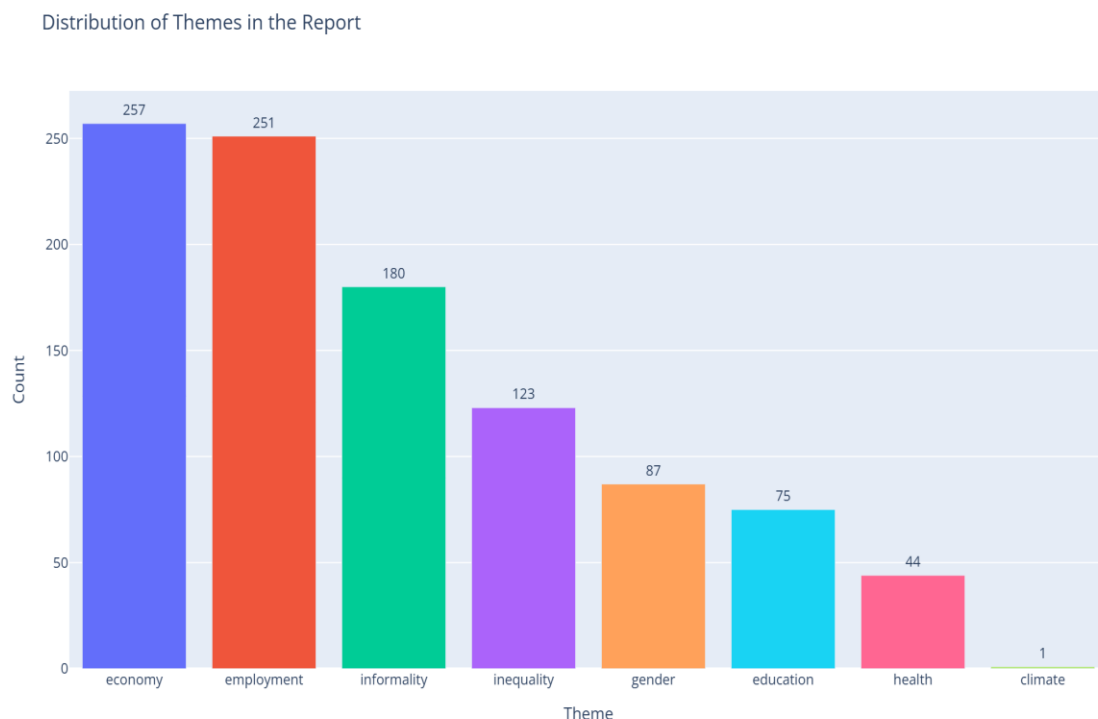


Figure 3. How often each theme came up across the whole report.

4. Development Trends

In addition to the headline statistics, the pipeline extracts any statistics that vary across years — for instance, HDI, life expectancy, and share of informal employment in different years — directly from sentences in the body text of the document. The statistics come through rather neatly because they tend to be expressed in a consistent format in sentences, e.g., "HDI increased to 0.82 in 2016" or "Life expectancy increased to 80 years in 2020."

Figure 4 shows six of those trends plotted against each other. As the values of these statistics span different orders of magnitude (the HDI ranges between 0 and 1 while population is measured in millions), we plot each trend separately on its own scale rather than putting them all on the same scale (the latter approach does not work because plotting HDI alongside population makes one of them illegible). We observe a strong and reasonable positive trend in life expectancy and share of GDP accounted for by informal economy. Two of the plots are quite noisy, however; for instance, the growth rate of the population sometimes jumps to unrealistic percentages. I think the most plausible explanation for this is that the extractor sometimes misidentifies some other

Demographic / Development Trends Over Time (each metric on its own scale)



Figure 4. Development trends over time, pulled straight from the report text.

5. Dashboard

Everything from the earlier steps comes together in one Streamlit dashboard an easy web application builder built on python that runs locally at `http://localhost:8501`. Rather than digging through separate JSON files by hand, the dashboard opens straight into the key numbers — HDI, informal employment rate, poverty risk, and GDP growth — right at the top, so the most important figures are visible immediately without clicking anywhere.

Below the headline numbers, the dashboard is organised into tabs: chapter summaries, themes and strengths/challenges, indicators, demographic trends, model comparison, and evaluation scores. Each tab pulls its data from the same set of JSON files the pipeline produces, so the dashboard always reflects whatever the latest pipeline run found. Figure 5 shows the dashboard open on the theme distribution tab, with the sidebar listing which model did which job.

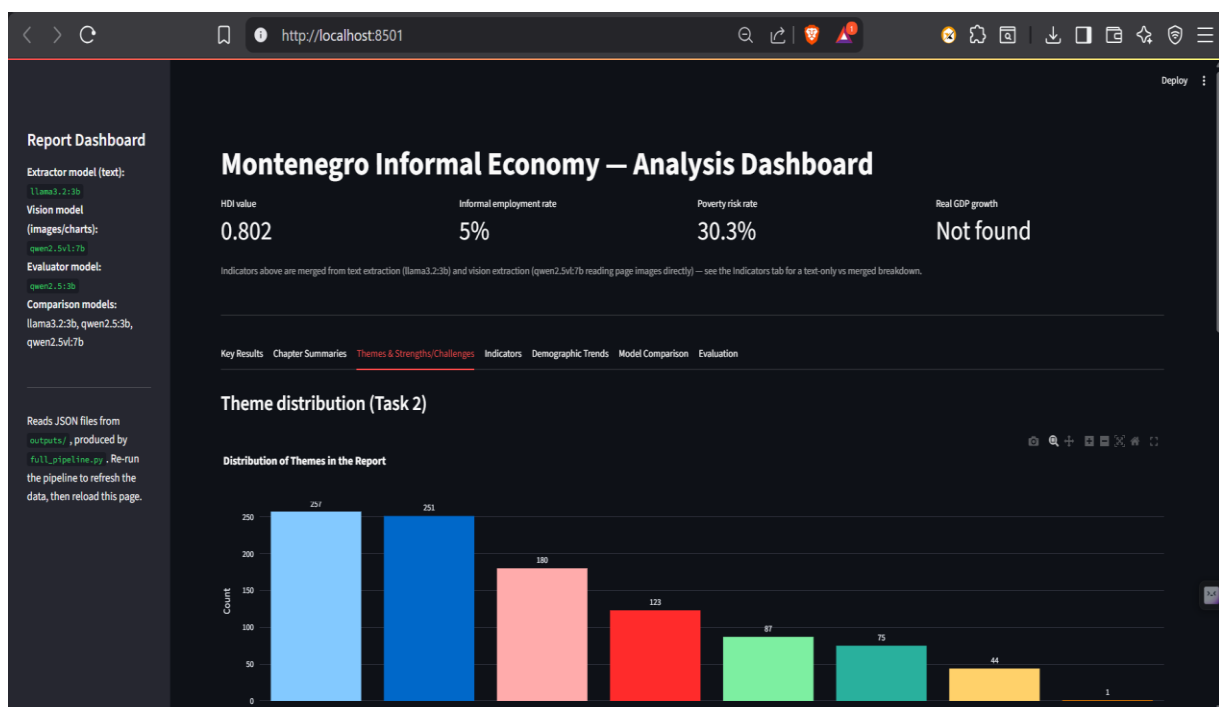


Figure 5. The Streamlit dashboard, running locally, showing the headline numbers and theme distribution tab.

Building this as a dashboard rather than just a report was worth the extra effort — it makes it much easier to spot when something looks off (like the population figure discussed in Section 3) at a glance, rather than needing to read through a JSON file line by line to notice a number is wrong.

6. Conclusion

Overall, this project shows that local LLMs can genuinely help turn a long policy report into structured, usable data, without needing a paid API or an internet connection. They're not perfect — a few numbers need manual double-checking, and combining a text model with a vision model doesn't automatically make results better; sometimes it just gives more chances for something to go wrong, as seen with the numbers discussed in Section 3.

That said, the strengths and challenges the models pulled out on their own, shown in Figure 6, line up well with what the report itself actually concludes high life expectancy and strong formal employment engagement on one side, and a high informal employment rate and gender gaps in employment on the other. That match is a good sign that even with its rough edges the overall approach is picking up the right signal or direction from the document, and a pipeline like this is a genuinely useful starting point for reading a long report quickly, as long as the final numbers get a quick human check before being used anywhere important.



Figure 6. Strengths and challenges pulled from the report, summarised in one view.

This type of RAG applications are very much useful in our everyday life because this pipelines and LLM's can reduce one week of work to just 3-4 hours of-course there could be no such accuracy but hope that upcoming fine-tuned LLM's also achieve that level of accuracy.

References

UNDP Montenegro. National Human Development Report: Informal Work in Montenegro. United Nations Development Programme.

Ollama (2025) Ollama Documentation. Available at: <https://ollama.com/>

Streamlit Inc. (2025) Streamlit Documentation. Available at: <https://streamlit.io/>